CENTRALESUPÉLEC
Control Engineering

CentraleSupélec

DATA ANALYTICS & MACHINE LEARNING

Forest Cover Type Classification

Mini-Project Report

PROJECT TEAM

Julia Ambert
Henrik Berger
Hélène Cucchi
Malek Wallier
Solal Wargon

COURSE INSTRUCTOR

Guillaume Sandou

1 Introduction

Forest cover type classification aims to predict the dominant forest cover type of a geographical area based on its environmental and cartographic characteristics. In this project, we use the Covertype dataset, available from the UCI Machine Learning Repository, which focuses on forest ecosystems in four wilderness areas of the Roosevelt National Forest in northern Colorado. It provides a large-scale multiclass classification problem involving seven forest cover types, based on geographical and environmental variables such as elevation, slope, distances to water and roads, sunlight exposure, and soil characteristics. The combination of different types of features, together with the potentially complex and nonlinear relationships between environmental conditions and forest cover, makes this dataset an interesting challenge for machine learning algorithms.

The main objective of this project is to develop a classifier that achieves the highest possible predictive performance on the Covertype dataset. To this end, several machine learning algorithms are implemented and evaluated. Different preprocessing techniques, dimensionality reduction through Principal Component Analysis (PCA), and hyperparameter optimization are investigated to identify the most effective configuration. The resulting models are assessed using appropriate classification metrics, while computational cost is also considered to evaluate the practicality of the proposed solutions.

2 Dataset and Problem Formulation

2.1 Dataset Description

The Covertype dataset contains 581,012 observations corresponding to 30 × 30 m forest land cells in the Roosevelt National Forest, Colorado. Each observation is associated with one of seven forest cover types, representing the dominant tree cover in the corresponding area. The objective is to predict this cover type using cartographic information, without relying on direct vegetation measurements [1].

Each observation is described by 54 input features, divided into three categories: 10 quantitative cartographic variables, 4 binary indicators representing wilderness areas, and 40 binary indicators representing soil types. The quantitative variables include elevation, slope, aspect, distances to hydrological features and roadways, and hillshade measurements. Table 1 summarizes the feature groups.

Feature family	Examples	Number
Quantitative	Elevation, slope, aspect, distances, hillshade	10
Wilderness areas	Binary area indicators	4
Soil types	Binary soil-type indicators	40

Table 1: Input feature groups in the Covertype dataset.

The dataset presents several challenges for classification, including heterogeneous feature types, differences in numerical scales, and unequal class frequencies. These characteristics motivate an exploratory analysis and the investigation of different preprocessing and classification strategies.

2.2 Mathematical Formulation

The problem is formulated as supervised multiclass classification. Let the dataset be defined as

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad \mathbf{x}_i \in \mathbb{R}^{54}, \quad y_i \in \{1, \dots, 7\}, \tag{1}$$

where $N = 581\,012$, $\mathbf{x}_i$ is the feature vector of observation i, and y_i is its corresponding forest cover type.

The objective is to construct a classification function

$$f : \mathbb{R}^{54} \longrightarrow \{1, \dots, 7\} \tag{2}$$

that accurately predicts the cover type of previously unseen observations.

Classification accuracy measures the proportion of correctly classified observations:

$$\text{Accuracy} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbb{1}\left(\hat{y}_i = y_i\right). \tag{3}$$

Here, $\hat{y}_i$ is the predicted class and y_i is the true class of observation i.

Because class frequencies differ substantially, macro-F1 is used as the primary model selection criterion, with accuracy and class-wise metrics providing complementary information.

3 Exploratory Data Analysis

3.1 Data Quality and Class Distribution

An initial exploration of the dataset was conducted to assess data quality and identify characteristics relevant to the classification task. No missing or infinite values were found, and no duplicated observations were detected. The binary encoding of wilderness areas and soil types was also verified, with exactly one active indicator per category for each observation.

As shown in Figure 1, the dataset is highly imbalanced. Cover types 1 and 2 account for approximately 85% of all observations, while cover type 4 represents less than 0.5%. In particular, a trivial classifier always predicting class 2 would already achieve an accuracy of approximately 48%.

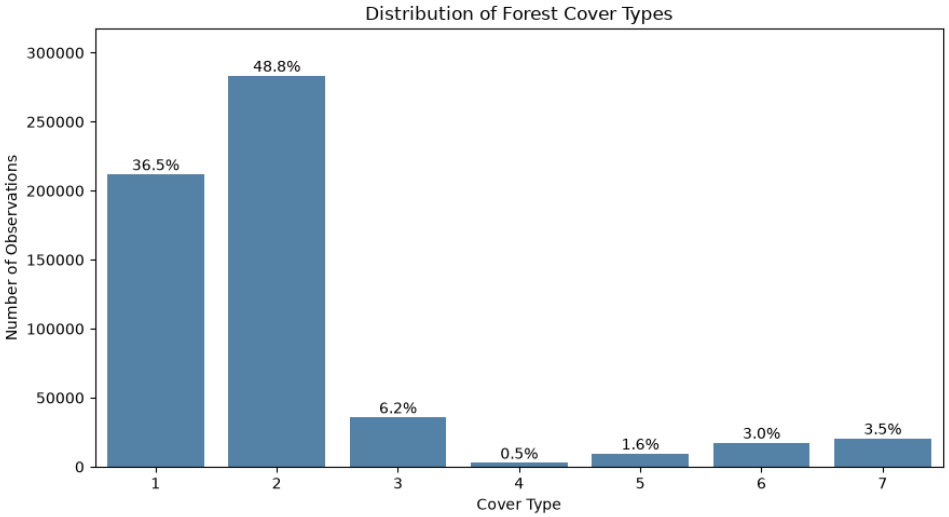


Figure 1: Distribution of observations across the seven forest cover types.

This imbalance highlights the limitations of using accuracy alone to evaluate classifier performance. Consequently, macro-averaged F1-score and class-wise recall will also be considered, giving greater visibility to performance on minority classes.

3.2 Feature Distributions and Relationships

The distributions of the numerical features were examined to identify potentially discriminative variables and relationships between features.

Elevation appears particularly informative, as illustrated in Figure 2. Cover types 1 and 7 are predominantly associated with higher elevations, whereas types 3, 4 and 6 are generally found at lower elevations. However, substantial overlap remains between certain classes, especially types 1 and 2, and types 3 and 6.

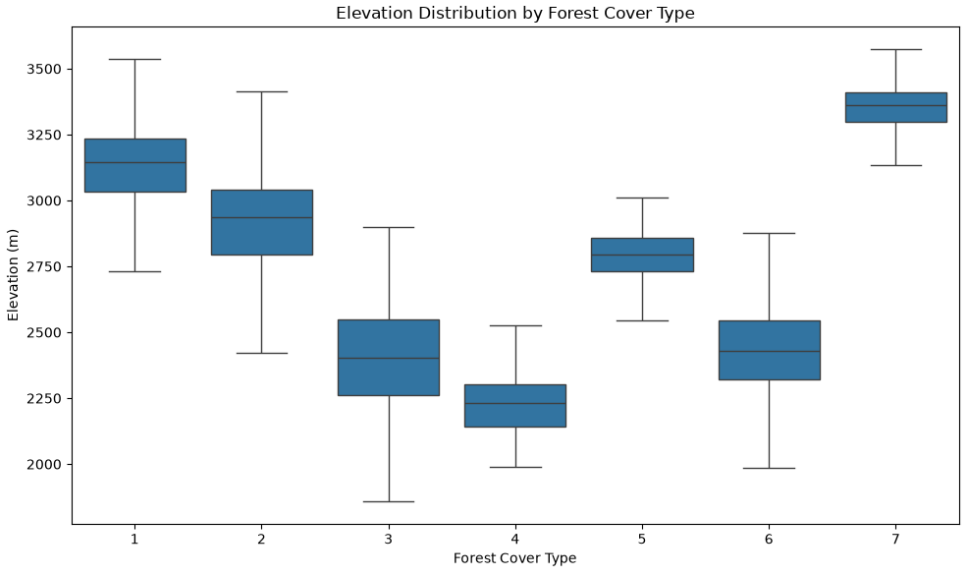


Figure 2: Elevation distributions for each forest cover type. Outliers are omitted for readability.

Other numerical variables, particularly slope and distances to roadways and fire ignition points, also exhibit class-dependent distributions. These observations suggest that combining multiple features is necessary to distinguish classes with overlapping individual distributions.

A correlation analysis further revealed some redundancy among the numerical features. In particular, Hillshade_9am

and Hillshade_3pm exhibit a strong negative correlation ($r = -0.78$), while horizontal and vertical distances to hydrology are positively correlated ($r = 0.61$).

Figure 3 presents the correlation matrix of the ten quantitative features. The observed correlations indicate that some variables contain partially redundant information, which motivates investigating dimensionality reduction through PCA.

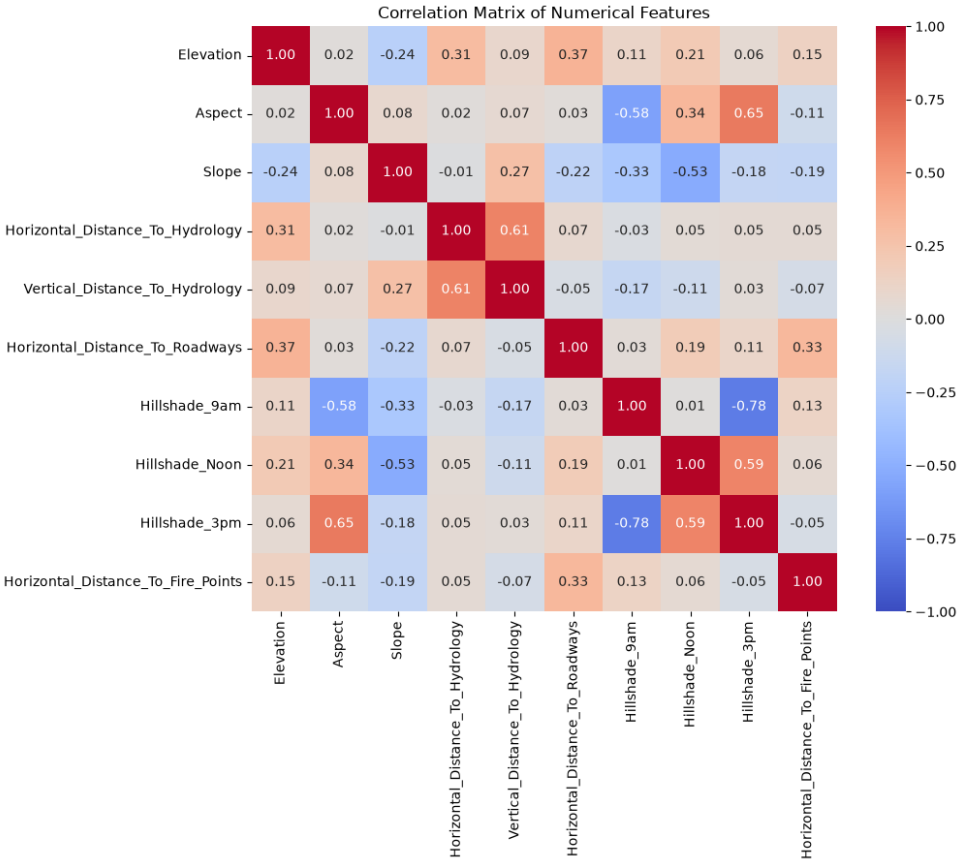


Figure 3: Correlation matrix of the ten quantitative features in the Covertype dataset.

3.3 Implications for Classification

Overall, the exploratory analysis highlights three important characteristics of the dataset: substantial class imbalance, heterogeneous feature distributions, and correlations between certain numerical variables.

These observations motivate the use of complementary evaluation metrics, the investigation of preprocessing techniques, and the comparison of different feature representations. In particular, the effect of PCA on classification performance will be assessed experimentally rather than inferred solely from the correlations between features.

4 Data Preparation and Dimensionality Reduction

4.1 Feature Preprocessing

Several preprocessing strategies were implemented to investigate their influence on classification performance. Data-dependent transformations are fitted exclusively on the training set to avoid data leakage, while the same transformations are applied to validation and test data.

Feature scaling. The numerical features exhibit substantially different scales. For algorithms sensitive to distances or feature magnitudes, standardization is considered:

$$\widetilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \tag{4}$$

where μ_j and σ_j are computed from the training set. Scaling is automatically applied before PCA, but remains optional when no dimensionality reduction is performed. In particular, tree-based models are generally insensitive to monotonic feature rescaling.

Rare soil types. The exploratory analysis revealed that several soil types are extremely uncommon, with six categories individually representing less than 0.1% of the observations. An optional preprocessing step was therefore implemented to remove soil-type indicators below a configurable frequency threshold.

However, rarity does not necessarily imply irrelevance: some uncommon soil types are strongly associated with specific forest cover classes. Removing them could therefore discard useful information, particularly for minority classes.

4.2 Cyclic Encoding of Aspect

A particularly important preprocessing consideration concerns the `Aspect` feature, which represents terrain orientation as an angle between 0° and 360°.

Although stored as a numerical variable, terrain orientation is inherently circular. For instance, orientations of 1° and 359° are physically separated by only 2°, despite having a numerical difference of 358°. Treating this feature as a linear variable therefore introduces an artificial discontinuity.

This issue is particularly visible for cover type 6, whose orientation distribution exhibits two apparent peaks near 0° and 360°. As illustrated in Figure 4, the circular representation reveals that these peaks actually correspond to a single continuous concentration of orientations around North.

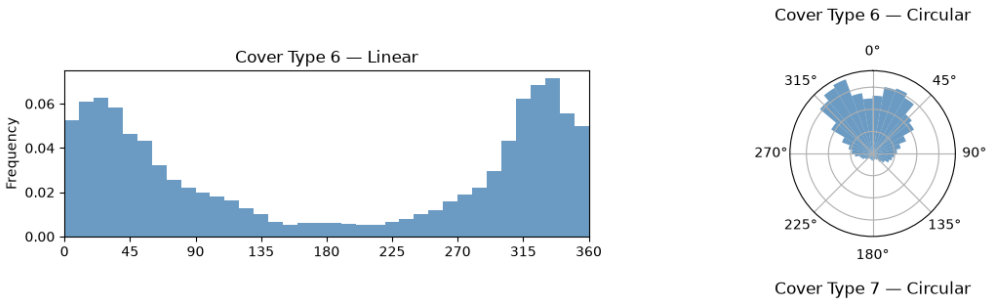


Figure 4: Linear and circular representations of terrain orientation for cover type 6.

To account for this periodicity, an alternative representation replaces the original angle θ with its sine and cosine:

$$\text{Aspect}_{\cos} = \cos\left(\frac{\pi\theta}{180}\right), \qquad \text{Aspect}_{\sin} = \sin\left(\frac{\pi\theta}{180}\right). \tag{5}$$

This transformation maps orientations onto the unit circle, preserving angular proximity and eliminating the artificial discontinuity at 0°/360°.

4.3 Principal Component Analysis

Principal Component Analysis (PCA) was investigated to reduce feature dimensionality and potential redundancy. It projects standardized data onto orthogonal directions of decreasing variance.

For the covariance matrix Σ of the standardized data, the cumulative explained variance retained by the first k principal components is

$$R_k = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{d} \lambda_j}, \tag{6}$$

where λ_j are the eigenvalues of Σ, ordered from largest to smallest.

Two PCA strategies were investigated:

- **Numerical PCA:** PCA is applied to the 10 standardized quantitative features, while the 44 binary indicators are preserved.
- **Global PCA:** PCA is applied to all 54 standardized features, including wilderness-area and soil-type indicators.

Figure 5 compares the cumulative explained variance for both approaches.



(a) Numerical PCA (b) Global PCA

Figure 5: Cumulative explained variance for PCA applied to numerical features only (a) and to all features (b).

For numerical PCA, the first three principal components explain approximately 65% of the variance. Retaining 7, 8 and 9 components preserves respectively 90%, 95% and 99% of the variance. At the 95% threshold, the total dimensionality only decreases from 54 to 52 features, suggesting limited potential computational benefits.

Global PCA requires approximately 40, 44 and 49 components to preserve the same respective variance thresholds. It therefore achieves greater dimensionality reduction, but also transforms the binary indicators and reduces their direct interpretability.

Furthermore, standardizing rare binary variables gives them greater weight in the PCA, potentially influencing the

resulting principal components.

Importantly, PCA maximizes explained variance rather than class separability. Consequently, neither approach can be considered preferable based on these observations alone. Their usefulness must be assessed experimentally through classification performance.

5 Classification Methods

Three classification algorithms were investigated: Random Forest (RF), K-Nearest Neighbors (KNN), and Multi-Layer Perceptron (MLP). These methods were selected to explore different approaches to the classification problem, with particular attention to the influence of preprocessing and hyperparameter tuning.

5.1 Random Forest

Random Forests were investigated for their ability to capture nonlinear relationships and interactions between heterogeneous features, without requiring feature scaling.

Beyond the standard hyperparameters (number of trees, maximum depth, minimum leaf size and feature subsampling), particular attention was given to class weighting to address the strong imbalance identified during the exploratory analysis.

The class weights were parameterized as

$$w_c = \left(\frac{N}{K N_c} \right)^{\alpha}, \tag{7}$$

where N_c denotes the number of training samples belonging to class c, $K = 7$ is the number of classes, and α controls the strength of the reweighting. This formulation allows a continuous adjustment between uniform weighting ($\alpha = 0$) and inverse-frequency weighting ($\alpha = 1$).

5.2 K-Nearest Neighbors

KNN was considered as a distance-based classification method, making it particularly relevant for evaluating the influence of feature scaling and dimensionality reduction.

The experiments focused on the number of neighbors k, the distance metric (Euclidean or Manhattan), and the weighting of neighbors. The effect of the different preprocessing configurations was also investigated.

5.3 Multi-Layer Perceptron

An MLP was investigated to model nonlinear relationships between the input features and the seven target classes.

The main parameters considered were the hidden layer architecture, regularization strength, and learning parameters. Particular attention was given to feature scaling, which directly affects neural network optimization.

5.4 Hyperparameter Optimization

For each classifier, different hyperparameter configurations were evaluated to improve predictive performance. The main parameters investigated are summarized in Table 2.

Model	Main hyperparameters investigated
Random Forest	Number of trees, maximum depth, minimum leaf size, feature subsampling, class weights
KNN	Number of neighbors, distance metric, neighbor weighting
MLP	Hidden layer architecture, regularization, learning parameters

Table 2: Main hyperparameters investigated for each classifier.

Hyperparameters were selected primarily using validation macro-F1, with accuracy as a complementary metric. The experimental protocol and results are detailed in the following sections.

6 Experimental Protocol

6.1 Data Splitting

The dataset was divided into three disjoint subsets using stratified random sampling: 70% for training, 15% for validation, and 15% for testing. A fixed random seed (42) was used to ensure reproducibility.

The training set was used to fit the classifiers, while the validation set was used for hyperparameter optimization and model selection. The test set was reserved for the final evaluation of the selected configurations. All preprocessing transformations requiring parameter estimation, including standardization and PCA, were fitted exclusively on the training data to avoid data leakage.

Once the optimal configurations had been selected, each classifier was retrained on the combined training and validation sets (85% of the dataset) before final evaluation on the test set.

6.2 Evaluation Metrics

Given the substantial class imbalance observed in the dataset, overall accuracy alone is insufficient to evaluate classification performance. We therefore use the macro-averaged F1-score as the primary model selection criterion.

For each class c, precision and recall are defined as

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{8}$$

where TP_c, FP_c, and FN_c denote respectively the true positives, false positives, and false negatives for class c, considering all other classes as negative.

The F1-score of class c is the harmonic mean of its precision and recall:

$$F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{9}$$

The macro-averaged F1-score is then obtained by averaging the class-specific F1-scores:

$$\text{MacroF1} = \frac{1}{7}\sum_{c=1}^{7} F1_c. \tag{10}$$

Unlike accuracy, macro-F1 gives equal importance to all seven classes, regardless of their frequency. It therefore provides a more balanced assessment of performance on minority and majority classes.

Accuracy and weighted F1-score are also reported as complementary metrics, together with class-wise precision, recall, and confusion matrices. Training and prediction times are measured to assess computational cost.

6.3 Experimental Procedure

For each classifier, an initial configuration was evaluated before investigating the influence of hyperparameters and preprocessing strategies. The experiments included feature scaling, PCA, and model-specific parameter optimization.

Random Forest hyperparameters were explored through a targeted grid search, while KNN and MLP configurations were investigated through dedicated experiments. All configurations were compared using the same validation set, with macro-F1 as the main selection criterion.

Additional experiments examined model complexity, feature importance, and the influence of training set size where relevant.

The final configurations were selected based on validation performance and subsequently evaluated on the common test set. Computational times were recorded using Python's `perf_counter`.

7 Results and Comparisons

7.1 Comparison of Classification Methods

After hyperparameter optimization, the three selected classifiers were retrained on the combined training and validation sets (493,860 observations) and evaluated on the common test set (87,152 observations).

Table 3 summarizes the final predictive performance of each model.

Model	Accuracy	Macro-F1	Weighted-F1
Random Forest	**0.96964**	**0.94851**	**0.96962**
KNN	0.96607	0.93844	0.96605
MLP	0.93970	0.90645	0.93959

Table 3: Final predictive performance on the test set.

Random Forest achieves the best overall predictive performance, with 96.96% accuracy and a macro-F1 of 0.9485. KNN obtains competitive results, with a macro-F1 of 0.9384, while MLP performs noticeably worse (0.9065).

The training and test inference times are respectively 52.68 s and 0.51 s for RF, 0.07 s and 480.98 s for KNN, and 706.76 s and 0.22 s for MLP.

However, overall metrics do not fully describe performance across the seven classes. Figure 6 compares class-specific F1-scores and recall values.

Random Forest achieves the highest F1-score for all seven classes, confirming that its superiority is not limited to the majority classes. The largest differences appear for some less represented classes, particularly class 5, where Random Forest reaches an F1-score of 0.9254, compared with 0.8924 for KNN and 0.8409 for MLP.

7.2 Error Complementarity Between Classifiers

Beyond their individual predictive performance, we investigated whether the three classifiers make errors on the same observations. For each pair of models, the overlap between their error sets was measured using the Jaccard similarity:

$$J(A, B) = \frac{|E_A \cap E_B|}{|E_A \cup E_B|}, \tag{11}$$

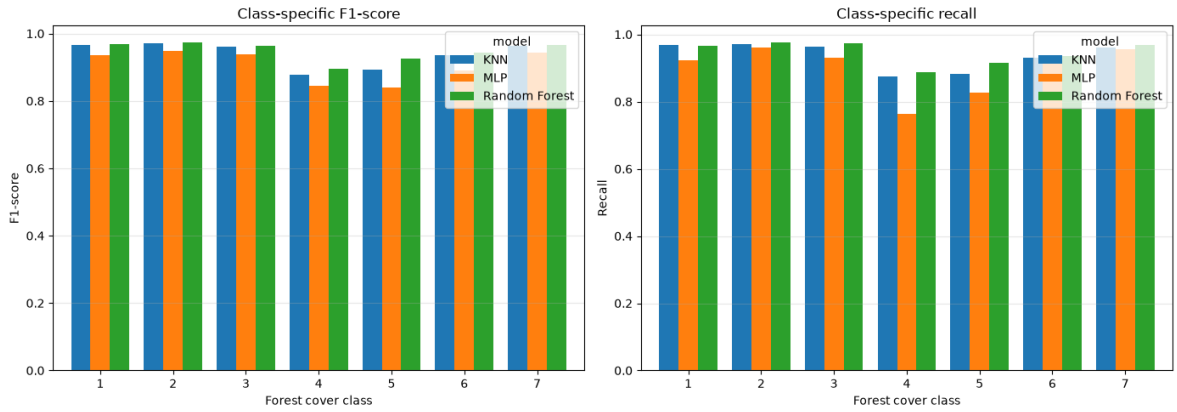


Figure 6: Class-specific F1-scores and recall for the three classifiers on the test set.

where E_A and E_B denote the sets of misclassified test observations for models A and B.

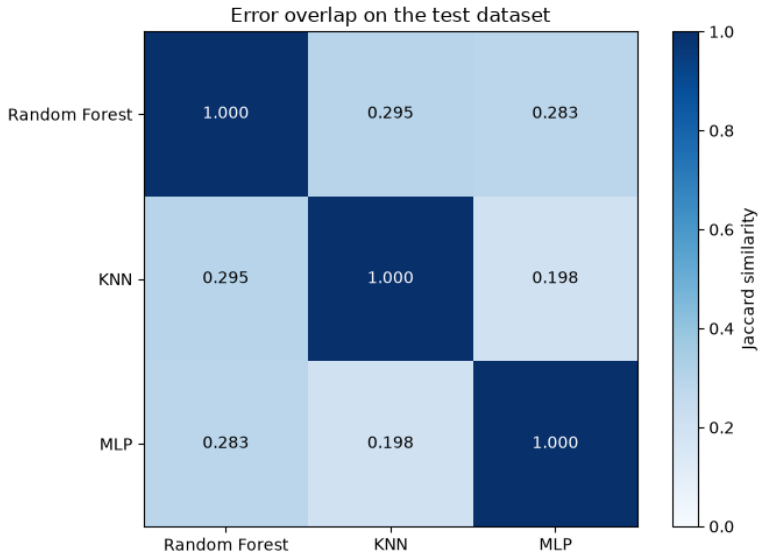


Figure 7: Similarity between the classification errors of Random Forest, KNN, and MLP on the test set.

The matrix highlights the extent to which the classifiers make similar or complementary errors.

Among the 87,152 test observations, only 905 are incorrectly classified by all three models. At least one classifier therefore predicts the correct class for 98.96% of the observations.

This suggests that the models capture partially complementary information. Combining their predictions could be an interesting direction for further improvement, although the performance of such an ensemble remains to be evaluated. The 98.96% figure is an oracle upper bound, not the performance of a tested ensemble.

7.3 Random Forest Optimization

Given its strong results, Random Forest was selected as the final classifier. Its hyperparameters were optimized through a targeted search over the number of trees, feature subsampling, and class-weighting strength.

Figure 8 illustrates the influence of these parameters on validation macro-F1 and computational cost.

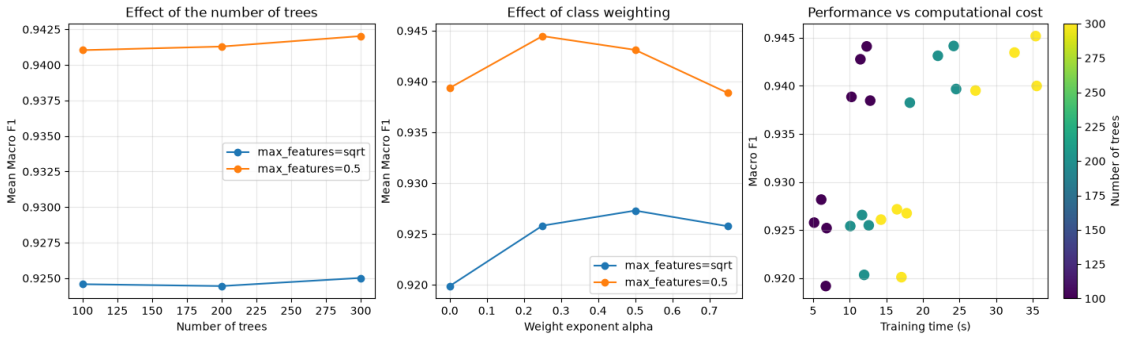


Figure 8: Influence of the number of trees, class weighting, and training cost on Random Forest validation performance.

The best configuration uses 300 trees, a feature subsampling ratio of 0.5, unlimited tree depth, a minimum leaf size of one sample, and a class-weighting exponent $\alpha = 0.25$.

It achieves a validation macro-F1 of 0.94517 and an accuracy of 0.96715.

The experiments show that moderate class weighting improves macro-F1, whereas stronger weighting does not necessarily lead to better results. Increasing the number of trees also improves performance, but with diminishing returns and increasing computational cost.

The influence of PCA was also investigated on the optimized Random Forest. Without PCA, the model achieves a validation macro-F1 of 0.94517. Numerical PCA reduces this score to 0.89751, while global PCA yields 0.91332.

Both transformations therefore degrade predictive performance and increase training time. The original feature representation was consequently retained for the final model.

7.4 Analysis of the Selected Random Forest

To investigate the remaining classification errors, Figure 9 presents the count and row-normalized confusion matrices of the optimized Random Forest on the validation set.

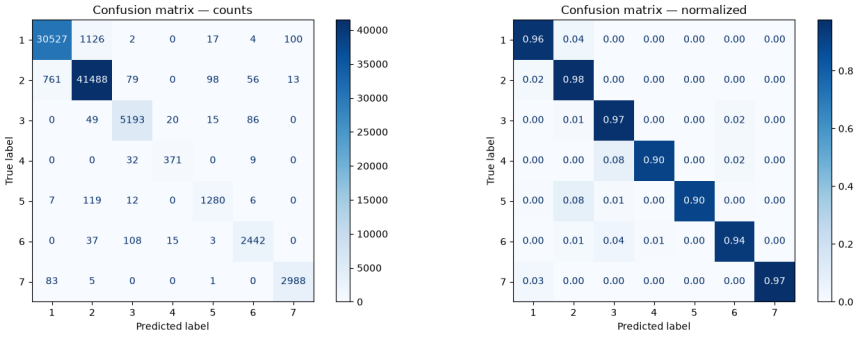


Figure 9: Count and row-normalized confusion matrices of the selected Random Forest on the validation set.

The classifier achieves strong performance across all seven classes. Nevertheless, classes 4 and 5 remain the most challenging, with validation F1-scores of 0.9071 and 0.9020, respectively.

The confusion matrix also reveals that some errors occur between classes with overlapping environmental characteristics, notably classes 1 and 2.

Despite these remaining errors, the relatively high F1-scores of the minority classes indicate that the model does not rely exclusively on the dominant classes to achieve its overall performance.

8 Discussion and Limitations

The experiments showed that Random Forest was the most effective approach for this dataset, both in terms of overall performance and classification of minority classes. One interesting result was that PCA actually reduced performance, despite the correlations observed between several features. This suggests that preserving the original feature representation is particularly useful for tree-based models. Similarly, the experiments on class weighting showed that moderate weighting was more effective than fully balancing the classes.

Although the results are encouraging, there are some limitations. The hyperparameter search covered only a limited number of configurations, and model evaluation relied on a single stratified split. Testing different random seeds would help determine how stable the results are. Moreover, since the dataset contains geographical observations, a random split does not necessarily reflect performance on entirely new geographical areas.

Finally, the comparison of classification errors revealed that the three models do not always make the same mistakes. Only 905 test observations were incorrectly classified by all three classifiers, suggesting that they capture somewhat different information. Exploring a combination of their predictions could therefore be an interesting next step, although its effectiveness would need to be tested.

9 Conclusion

The objective of this project was to develop the best possible classifier for the Covertype dataset by exploring different machine learning methods and preprocessing strategies. Among Random Forest, KNN and MLP, Random Forest achieved the best results, reaching 96.96% accuracy and a macro-F1 score of 0.9485 on the test set. It also performed well across all seven forest cover types, despite the significant class imbalance.

Beyond the final scores, this project highlighted the importance of understanding the data and testing different model configurations. Feature engineering, hyperparameter tuning and class weighting all played a role in our experiments, while more complex preprocessing techniques such as PCA did not necessarily lead to better results. Although Random Forest proved to be the strongest model among those tested, the analysis of classification errors suggests that further improvements could still be explored.

References

[1] UCI Machine Learning Repository, *Covertype Dataset*. https://archive.ics.uci.edu/dataset/31/covertype.

User Guide: Running and Modifying the Experiments

Installation and Setup

The project requires Python 3.12 or later. From the repository root, install the package and its dependencies using:

```
python -m pip install -e .
```

This installs the required libraries, including NumPy, pandas, scikit-learn, Matplotlib, Seaborn, and Jupyter's Python kernel. The two notebooks can then be opened and executed in VS Code or Jupyter using the corresponding Python environment.

Data Loading and Utilities

The `Utils/Utils.py` module provides the shared functions for data loading, splitting, and preprocessing.

The dataset is loaded using:

```
from Utils.Utils import load_covertype_data
X, y = load_covertype_data(flatten_target=True)
```

If local Parquet files are available in `DATA/raw/`, they are loaded directly. Otherwise, the dataset is automatically downloaded from the UCI repository and cached locally for subsequent runs.

The `split_data()` function creates training, validation, and test sets. By default, it uses stratified sampling with proportions 70%/15%/15% and random seed 42.

Running the Notebooks

The project is organized around two notebooks:

- `DATA/data_explore.ipynb`: dataset exploration, class distributions, feature analysis, preprocessing transformations, and PCA visualizations.
- `experiments.ipynb`: training, hyperparameter optimization, and evaluation of Random Forest, KNN, and MLP classifiers, followed by their final comparison.

Run the notebooks from top to bottom, starting with data exploration if desired. The experiments notebook can also be executed independently, as it loads and splits the data using the shared utilities. Results and figures are displayed directly in the notebooks. Some experiments, particularly MLP training and hyperparameter searches, can be time-consuming. We therefore recommend using the saved notebook outputs to inspect the results rather than rerunning every experiment. Individual cells can be executed when testing specific configurations.

Changing Parameters

Preprocessing options are configured through `PreprocessingConfig` in `Utils/Utils.py`. For example:

```
from Utils.Utils import (
    PreprocessingConfig, build_preprocessor
)

config = PreprocessingConfig(
    scaling=True,
    cyclic_aspect=True,
    pca="numeric",
    pca_components=0.95
)
preprocessor = build_preprocessor(config)
```

The PCA mode can be set to `"none"`, `"numeric"`, or `"all"`. The `pca_components` parameter accepts either a number of components or a retained variance ratio. Rare soil filtering can also be enabled using `rare_soil_threshold`.

Model hyperparameters can be modified directly in `experiments.ipynb`, particularly in the *Hyperparameter Optimization* sections of each classifier. For example, the Random Forest search grid controls the number of trees, feature subsampling, and class-weighting exponent.

After changing a configuration, rerun the relevant cells and their dependencies. Model configurations are compared primarily using validation macro-F1, while the final test evaluation is performed after model selection.